

University of Central Punjab
Faculty of Information Technology
Object Oriented Programming
Spring 2025

Lab 06	
Topic	Classes in C++, access specifiers, objects, methods, constructors, destructors, Constructor Initializer List, Static Objects , array of objects, pointer to objects, array of pointers to objects, Friend functions and friend classes
Objective	Making students familiarize with the concepts of constructors, destructors, array of objects, pointer to objects, array of pointers to objects, Friend functions and friend classes
CLO	Apply object-oriented programming principles to implement real-world problems.

Instructions:

- Indent your code.
- Comment your code.
- Use meaningful variable names.

- Plan your code carefully on a piece of paper before you implement it. •
Name of the program should be same as the task name. i.e. the first program should be Task_1.cpp
- **void main() is not allowed. Use int main()**
- **You are not allowed to use any built-in functions**
- **You are required to follow the naming conventions as follow:**
- **Variables:** firstName; (no underscores allowed)
- **Function:** getName(); (no underscores allowed)
- **ClassName:** BankAccount (no underscores allowed)

Students are required to complete the following tasks in lab timings.

Task 1: Static Object

Objective:

Demonstrate how a **static object** retains its state between multiple function calls.

Problem Statement:

Create a class Counter with a static object to persist data across function calls.

Attributes:

- int count

Functions:

- **Default Constructor:** Initializes count = 0 and prints "Counter initialized."
- **void increment():** Increments count by 1.
- **void displayCount():** Displays current count.
- **Destructor:** Prints "Counter destroyed."

Sample Code:

```
void performTask() {
    static Counter c; // Static object
    c.increment();
    c.displayCount();
}
```

Sample Output:

```
First call to performTask():  
Counter initialized.  
Current count: 1
```

```
Second call to performTask():  
Current count: 2
```

```
Third call to performTask():  
Current count: 3
```

```
Program ending...  
Counter destroyed.
```

Task 2: Arrays of Objects

Objective:

Create a class Student with data members name and marks. Write a program to create an array of 5 Student objects and input/output their details.

```
o char* name (name of the student) o  
int marks (marks of the student)
```

Required Functions:

- **Default Constructor:**
 - o Initialize name to "Unknown", marks to 0.
- **Parameterized Constructor:** o Initialize name and marks using the constructor initializer list.
- **Destructor:**
 - o Deallocate dynamic memory for name.
- **Setter and Getter functions:**
 - o Implement setters for name and marks.
 - o Implement getters to retrieve name and marks.
- **Display function:**
 - o Implement a display function void display () that displays the name and marks of the student.

Main Function:

- o Create array of 5 `student` objects and input & output their details in `main ()`.

SAMPLE OUTPUT

Enter name of student: Alice

Enter marks of student: 78

Enter name of student: Bob

Enter marks of student: 92

Enter name of student: Charlie

Enter marks of student: 85

Enter name of student: Diana

Enter marks of student: 69

Enter name of student: Ethan

Enter marks of student: 74

Student Details:

Name: Alice, Marks: 78

Name: Bob, Marks: 92

Name: Charlie, Marks: 85

Name: Diana, Marks: 69

Name: Ethan, Marks: 74

Task3: Pointers to Objects

Objective:

Define a class Book with members title and price. Create a pointer to a Book object and use it to input and display the book details.

- o `char* title` (title of the book)
- o `float price` (price of the book)

Required Functions:

- **Default Constructor:**
 - o Initialize name to "Unknown", price to 0.0
- **Parameterized Constructor:** o Initialize `name` and `price` using the constructor initializer list.
- **Destructor:**
 - o Deallocate memory for `title`.
- **Member Function:**
 - o Implement a display function `void display ()` that displays the title and price of the book.

Main Function:

- o In `main ()`, declare a `Book*` pointer.
- o Dynamically allocate a Book object by invoking the parameterized constructor.
- o Prompt the user to enter the book's title and price, read these values into the dynamically allocated object via the pointer, then call its `display ()` method to show the details.
- o Finally, deallocate the object before exiting.

SAMPLE OUTPUT

/Enter title of book: The Alchemist

Enter price of book: 19.99

Book Details:

Title: The Alchemist

Price: 19.99

Task 4: Array of Pointers to Objects

Objective:

Design a `Employee` class that the following attributes. Write a program that uses an array of pointers to `Employee` objects to store and display information for 3 employees.

- o `int id` (employee id)
- o `double salary` (salary of the employee)

Required Functions:

- **Default Constructor:**
 - o Set a unique `id` to 001, `salary` to 1000.10.
- **Parameterized Constructor:** o Initialize `id` and `salary` using the constructor initializer list.
- **Setter and Getter Methods:**
 - o Implement setter and getter functions for `id` and `salary`.

Main Function:

- o Declare an array of pointers.
- o Dynamically allocate an `Employee` object
- o For each pointer in the array call `display()`
- o Loop over the array and delete before exiting

Task 5: Friend Functions and Friend Classes

Objective:

Create two classes `Rectangle` and `Calculator`. Make a member function of `Calculator` a friend of `Rectangle` to calculate and display the area of the rectangle. Also make `Calculator` a friend class of `Rectangle`.

Class `Rectangle`:

- o `int length, width`
- o `Rectangle(int l, int w);`
- o friend `int calculateArea(const Rectangle& r);`
- o friend class `Calculator;`

Class `Calculator`:

- o `int area(const Rectangle& r);`

Required Functions:

- **Parameterized Constructor:** o Initialize `length` and `width` using the constructor initializer list.
- **Member Functions:**
 - o Declare `int calculateArea(const Rectangle& r)` as a friend inside `Rectangle` so it can read `length` and `width` directly.
 - o Declare class `Calculator` as a friend of `Rectangle`
 - o In `Calculator`, implement `int area (const Rectangle& r)`

Main Function:

- o Create a `Rectangle` instance with specific dimensions.
- o Call the standalone friend function and display its return value as the area.
- o Instantiate a `Calculator` object, call its `area` method on the same `Rectangle`, and display that result.
- o Exit the program

Task 6: Menu Based

Now create a menu-based program where user should select the function that needs to be tested from the above functions.

For example:

- Press 1 for testing Pointer to Objects.
- Press 2 for testing Array of Pointers to Objects.
- Press 3 for testing Friend functions and friend classes.
- Press 4 for exit.